

AI-SA-IN-TH-000017

Module 8.1 · LLM Evaluation Fundamentals · AI Safety: Evaluation, Guardrails & Responsible Deployment

Hallucination Detection Methods

Self-Consistency, SelfCheckGPT, Semantic Entropy, NLI Scoring, and RAGAS

9 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

Theory

Domain

Learning Objectives

- 1 Explain why hallucination detection requires dedicated tools beyond standard output monitoring
- 2 Describe the self-consistency checking algorithm and calculate a consistency score
- 3 Explain how SelfCheckGPT uses multiple samples to detect hallucinations without reference answers
- 4 Define semantic entropy and explain how it quantifies model uncertainty about factual claims
- 5 Apply NLI-based factual consistency scoring to classify an output as entailed, neutral, or contradicted
- 6 Interpret a RAGAS evaluation report and identify which metrics signal hallucination risk

Why Dedicated Detection Is Necessary

Hallucination Detection Gap

Standard output monitoring (latency, error rates, user satisfaction) cannot detect hallucinations: a hallucinatory response returns HTTP 200, generates no exception, and may receive positive user feedback from users who do not know the answer is wrong. Dedicated detection requires semantic understanding, not infrastructure metrics.

- Invisibility to standard monitoring: hallucinations are syntactically and semantically valid outputs — no error signal in application logs
- User feedback is unreliable: users in domains outside their expertise (medical, legal, scientific) cannot evaluate factual accuracy — they may rate confident wrong answers positively
- Ground-truth-free detection: the most important property for production systems — we need detection that works without a pre-existing answer key
- Three detection paradigms: (1) internal consistency (compare model with itself), (2) external consistency (compare with retrieved evidence), (3) calibration assessment (model's own confidence scores)
- When to use which: internal for open-domain; external for RAG systems; calibration for monitoring pipelines

Detection Method Taxonomy

Internal Consistency

- 1 Self-Consistency Checking
(majority vote over N samples)
- 2 SelfCheckGPT (NLI + n-gram + BERTScore)
- 3 Semantic Entropy (clustering semantics)

External Consistency

- 1 NLI Factual Consistency
(entailment vs context)
- 2 RAGAS Faithfulness
(claim-level grounding check)
- 3 Citation Verification
(source attribution check)

Calibration Methods

- 1 Confidence Scoring
(logprob-based routing)
- 2 LLM-as-Judge
(GPT-4 hallucination scorer)
- 3 ECE Monitoring
(production calibration drift)

Use Internal when no ground truth exists (open-domain) · External when context documents are available (RAG) · Calibration for production monitoring.

Self-Consistency Checking

Self-Consistency (Wang et al., 2022)

Self-consistency sampling generates N independent responses to the same question using non-zero temperature, then measures agreement across responses. High agreement $\rightarrow$ likely correct; low agreement $\rightarrow$ likely uncertain or hallucinating. Requires no reference answer.

- Algorithm: (1) sample N responses (typically 5–20) at temperature 0.7–1.0; (2) extract the key factual claim from each; (3) compute pairwise agreement via string match, NLI, or BERTScore; (4) flag low-agreement responses for review
- Majority vote: for factual QA, take the answer that appears in the largest fraction of samples — improves accuracy by 10–15pp over greedy decoding
- Consistency score: fraction of sample pairs that agree on the key claim — score < 0.6 signals high hallucination risk
- Cost: N API calls per query — typically 5 samples; balance between detection power and cost
- Limitation: if the model consistently hallucinates the same wrong answer, self-consistency gives false confidence — most effective for detecting uncertain/variable hallucinations, not systematic ones

SelfCheckGPT — Algorithm Walkthrough

Python (concept)

```
# SelfCheckGPT (Manakul et al., 2023) – three scoring modes
from selfcheckgpt.modeling_selfcheck import SelfCheckNLI
import anthropic, torch

nlp_check = SelfCheckNLI(nli_model='cross-encoder/nli-deberta-v3-large')
client = anthropic.Anthropic()

def selfcheck_score(question: str, n_samples: int = 5) -> dict:
    # Step 1: Generate N samples at temperature > 0
    samples = [client.messages.create(
        model='claude-haiku-4-5-20251001', max_tokens=512,
        messages=[{'role': 'user', 'content': question}],
        temperature=0.8
    ).content[0].text for _ in range(n_samples)]

    # Step 2: Pick one response as 'passage', rest as 'samples'
    passage, evidence = samples[0], samples[1:]

    # Step 3: NLI-based consistency score (lower = more hallucination)
    scores = nlp_check.predict(sentences=[passage], sampled_passages=evidence)
    return {'consistency_score': 1 - float(scores[0]), 'n_samples': n_samples}
```

consistency_score near 0 = high hallucination risk; near 1 = high consistency. Threshold 0.6: flag for review if below.
SelfCheckGPT achieves ~0.85 AUC on detecting hallucinations in GPT-3 Wikipedia generation.

Semantic Entropy — Worked Calculation

$$\text{Semantic Entropy} = -\sum P(\text{meaning_cluster_k}) \cdot \log P(\text{meaning_cluster_k})$$

1. Generate 5 samples for Q: 'When was the first iPhone released?'
2. Samples: ['2007', '2007', '2007', '2008', '2007']
3. Cluster by semantics: {2007: 4 samples, 2008: 1 sample}
4. $P(\text{cluster_2007}) = 4/5 = 0.80$, $P(\text{cluster_2008}) = 1/5 = 0.20$
5. Semantic Entropy = $-(0.80 \times \log_2(0.80) + 0.20 \times \log_2(0.20))$
6. $= -(0.80 \times (-0.322) + 0.20 \times (-2.322)) = -(-0.258 - 0.464) = 0.722$
7. Interpretation: entropy = 0.722 — low (correct answer dominates). If entropy > 1.5, flag as uncertain.

Note: Semantic entropy is preferred over lexical consistency because paraphrases cluster correctly: 'nineteen zero seven' and '2007' are the same semantic cluster despite different tokens.

NLI-Based Factual Consistency Scoring

Natural Language Inference (NLI) for Hallucination Detection

NLI models classify the relationship between a premise (retrieved context) and a hypothesis (model output) into three categories: entailed (context supports claim), contradicted (context refutes claim), or neutral (context neither supports nor refutes). Used to detect faithfulness hallucinations in RAG systems.

- Entailment: 'The Eiffel Tower is 330m tall' — if context says 'Eiffel Tower height: 330 metres' → ENTAILED (not hallucinated)
- Contradiction: model says 'Einstein won the Nobel Prize in Physics in 1922' — context says '1921' → CONTRADICTED (hallucination)
- Neutral: model says 'Einstein enjoyed long walks' — context says nothing about his habits → NEUTRAL (may be correct but unverified)
- Faithfulness score: fraction of model claims that are ENTAILED by retrieved context — target > 0.90 for high-stakes deployments
- Models: DeBERTa-v3-NLI achieves 92% accuracy on SNLI; cross-encoder/nli-deberta-v3-large is the best performing open-weight NLI model

NLI Consistency Pipeline

Python

```
from transformers import pipeline

nli_pipe = pipeline('zero-shot-classification',
                    model='cross-encoder/nli-deberta-v3-large')

def nli_faithfulness_score(response: str, context: str) -> dict:
    """Split response into sentences, check each against context."""
    import nltk
    sentences = nltk.sent_tokenize(response)
    results = []
    for sent in sentences:
        out = nli_pipe(sent, candidate_labels=['entailment', 'neutral', 'contradiction'],
                       hypothesis_template={},
                       multi_label=False)
        label = out['labels'][0]
        score = out['scores'][0]
        results.append({'sentence': sent, 'label': label, 'confidence': round(score, 3)})
    n_ent = sum(1 for r in results if r['label'] == 'entailment')
    faithful = n_ent / len(results) if results else 0
    return {'faithfulness': round(faithful, 3), 'sentences': results}
```

Faithfulness < 0.80: high hallucination risk, route to human review. Faithfulness < 0.50: block response. NLI inference: ~40 ms per sentence on CPU; vectorise with batching for production throughput.

RAGAS — Standard RAG Hallucination Evaluation Framework

RAGAS (Retrieval Augmented Generation Assessment)

RAGAS is an open-source framework that provides a reference-free evaluation suite for RAG systems, measuring four key dimensions that collectively capture hallucination risk, retrieval quality, and answer relevance — using an LLM judge to perform fine-grained claim-level assessment.

- Faithfulness: fraction of answer claims that are supported by retrieved context — primary hallucination metric (target > 0.90)
- Answer Relevancy: how well the answer addresses the question — measures whether the model answered what was asked
- Context Precision: fraction of retrieved context chunks that are actually relevant to the question — measures retrieval quality
- Context Recall: fraction of ground-truth information that is covered by the retrieved context — measures retrieval completeness
- Reference-free: RAGAS uses an LLM (GPT-4 or Claude) to decompose answers into atomic claims and verify each against context — no human-written reference answers required
- Usage: run RAGAS on every model version before production deployment; track faithfulness score as a key reliability KPI

RAGAS Metrics — Definitions and Interpretation

Metric	Formula (Conceptual)	Range	Target	What Low Score Means
Faithfulness	Entailed claims / Total claims in answer	0–1	≥ 0.90	Model hallucinating beyond retrieved context — grounding failure
Answer Relevancy	Similarity(answer, question) via embeddings	0–1	≥ 0.85	Model answered off-topic or partially addressed the question
Context Precision	Relevant chunks / Total retrieved chunks	0–1	≥ 0.75	Retriever returning too much irrelevant context — noisy RAG
Context Recall	Ground-truth facts covered / All ground-truth facts	0–1	≥ 0.80	Retriever missing key information — incomplete RAG coverage

LLM-as-Judge for Hallucination Detection



Judge Prompt (Claim Extractor):

System: Extract all atomic factual claims from the answer below.
Return as a JSON list of strings.

Answer: {model_response}

System: Is this claim supported by the context? Answer ONLY:
'yes' or 'no'

Context: {retrieved_context}
Claim: {claim_i}

$$\text{Faithfulness} = \text{count}(\text{yes}) / \text{count}(\text{all claims})$$

Claim Extraction and Verification — Worked Example

RAGAS faithfulness evaluation on a medical RAG response

Question + Retrieved Context

Q: 'What is the standard first-line treatment for Type 2 diabetes?'

Context: 'Metformin is recommended as the first-line pharmacological treatment for Type 2 diabetes by both ADA and EASD guidelines. It is generally well tolerated and has a low risk of hypoglycaemia.'

Model Response + Claim Verification

Response: 'The first-line treatment for Type 2 diabetes is Metformin. It is recommended by ADA guidelines and approved by the FDA in 1994. Patients typically start at 500mg twice daily, which reduces HbA1c by approximately 1.5%.'

Extracted claims:

- Claim 1: 'Metformin is first-line for T2D' → ENTAILED by context ✓
- Claim 2: 'ADA recommends Metformin' → ENTAILED by context ✓
- Claim 3: 'FDA approved Metformin in 1994' → NOT IN CONTEXT → NEUTRAL ⚠
- Claim 4: '500mg twice daily' dosage → NOT IN CONTEXT → NEUTRAL ⚠
- Claim 5: 'Reduces HbA1c by ~1.5%' → NOT IN CONTEXT → NEUTRAL ⚠

Faithfulness = $2/5 = 0.40$ → HALLUCINATION DETECTED — 3 claims ungrounded

Detection Method Comparison

Method	Requires Reference?	Latency	Accuracy (AUC)	Best For
Self-Consistency (N=5)	No	5× query cost	0.79	Open-domain factual QA; uncertain claims
SelfCheckGPT (NLI mode)	No	5× + NLI inference	0.85	General-purpose; sentence-level detection
Semantic Entropy	No	5× query cost	0.82	Uncertainty quantification; routing decisions
NLI Faithfulness	Context doc (not answer)	~40ms / sentence	0.88	RAG systems; document-grounded responses
RAGAS Faithfulness	Context doc (not answer)	~2 LLM calls	0.87	RAG evaluation; CI/CD reliability pipeline
LLM-as-Judge (GPT-4o)	Context doc	1 LLM call	0.91	High-stakes final validation; nuanced claims

Case Study — Production Hallucination Monitoring

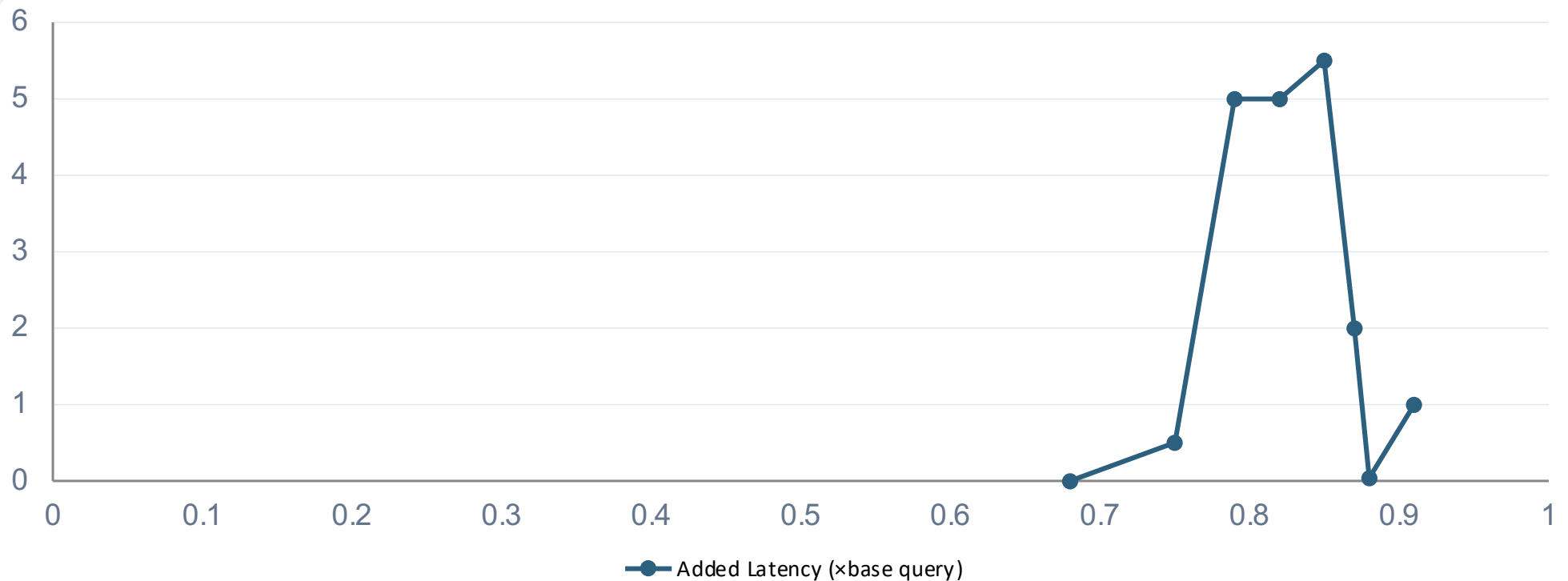
Case: Enterprise Legal Research Assistant — Vectara Hallucination Leaderboard (2024)

- A law firm deployed a RAG-based legal research assistant to help attorneys find relevant case law — 200+ daily queries, 10 attorneys
- Week 1 post-launch: attorneys flagged 3 responses with incorrect citations. Initial investigation found the issue intermittent and hard to reproduce
- Root cause: faithfulness monitoring was not implemented — the pipeline measured latency and error rates but not factual accuracy
- Resolution: integrated RAGAS faithfulness evaluation on every response (< 100 ms overhead using async Claude haiku judge)
- Outcome: baseline faithfulness score = 0.71 — far below the 0.90 target. Identified retrieval configuration error causing context to often not contain the relevant legal text. Fixed vector store chunking strategy — faithfulness rose to 0.93 within 2 weeks

Key Lesson

Key lesson: hallucination rate is a mandatory production metric, not optional. Without faithfulness monitoring, this system silently hallucinated citations for weeks — only caught by frustrated attorneys. RAGAS faithfulness check added \$0.008 per query — 99.6% cheaper than the attorney time spent verifying bad outputs.

Detection Method — AUC Accuracy vs Added Latency Cost



Left to right: No detection, Log-prob, Self-Consist(N=5), Semantic Entropy, SelfCheckGPT, RAGAS, NLI(sync), LLM-Judge.

Recommended production stack: NLI faithfulness (fast, 0.88 AUC) for inline screening + RAGAS in CI/CD + LLM-Judge for sampled audit.

Key Takeaways

AI-SA-IN-TH-000017

- 1 Standard infrastructure monitoring cannot detect hallucinations — dedicated semantic detection is required because hallucinations return HTTP 200, generate no exceptions, and may receive positive user ratings
- 2 Self-consistency sampling generates N responses and measures agreement — effective for open-domain factual uncertainty; consistency < 0.6 signals high hallucination risk
- 3 SelfCheckGPT combines NLI, n-gram, and BERTScore consistency across N samples — best all-around hallucination detector without requiring reference answers (AUC ~0.85)
- 4 NLI-based faithfulness scoring classifies each response claim as entailed/neutral/contradicted by context — the preferred method for RAG systems; faithfulness < 0.80 should trigger human review
- 5 RAGAS provides a comprehensive four-metric RAG reliability report (faithfulness, answer relevancy, context precision, context recall) — run it on every model version before production deployment and track faithfulness as a mandatory KPI